FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA

Maria Luiza Alves De Aquino – 561802 - 2TDSPW

João Victor Gomes de Souza – 560907 - 2TDSPW

Lucas Barranha Giannini – 564508 - 2TDSPW

Maicon Douglas da Silva Timoteo – 561279 - 2TDSPW

Documentação do Projeto Genius(IOT)

São Paulo
2025

FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA

Maria Luiza Alves De Aquino – 561802 - 2TDSPW

João Victor Gomes de Souza – 560907 - 2TDSPW

Lucas Barranha Giannini – 564508 - 2TDSPW

Maicon Douglas da Silva Timoteo – 561279 - 2TDSPW

Genius(IOT)

CP1 apresentado à Faculdade de Informática
e Administração Paulista como requisito de nota
para a avaliação da disciplina IOT, sob a orientação do Professor
Emil Freme

São Paulo

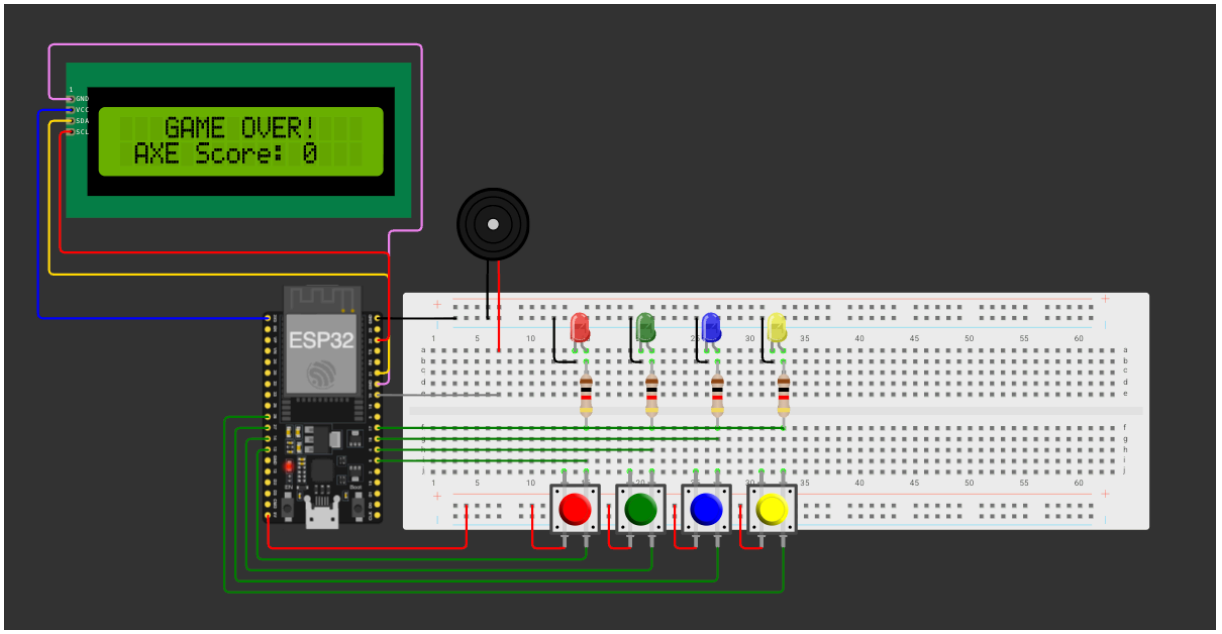
2025

Objetivo do projeto

Transformar o jogo “Genius” que fizemos nas aulas anteriores em um dispositivo IoT conectado, capaz de identificar o jogador, aumentar a dificuldade progressivamente e registrar as pontuações em um ranking global em tempo real.

O processo possui 4 fases:

1.Interface de identificação: Parte de criação do Hardware. Criamos nosso projeto de arduino pelo Wokwi.



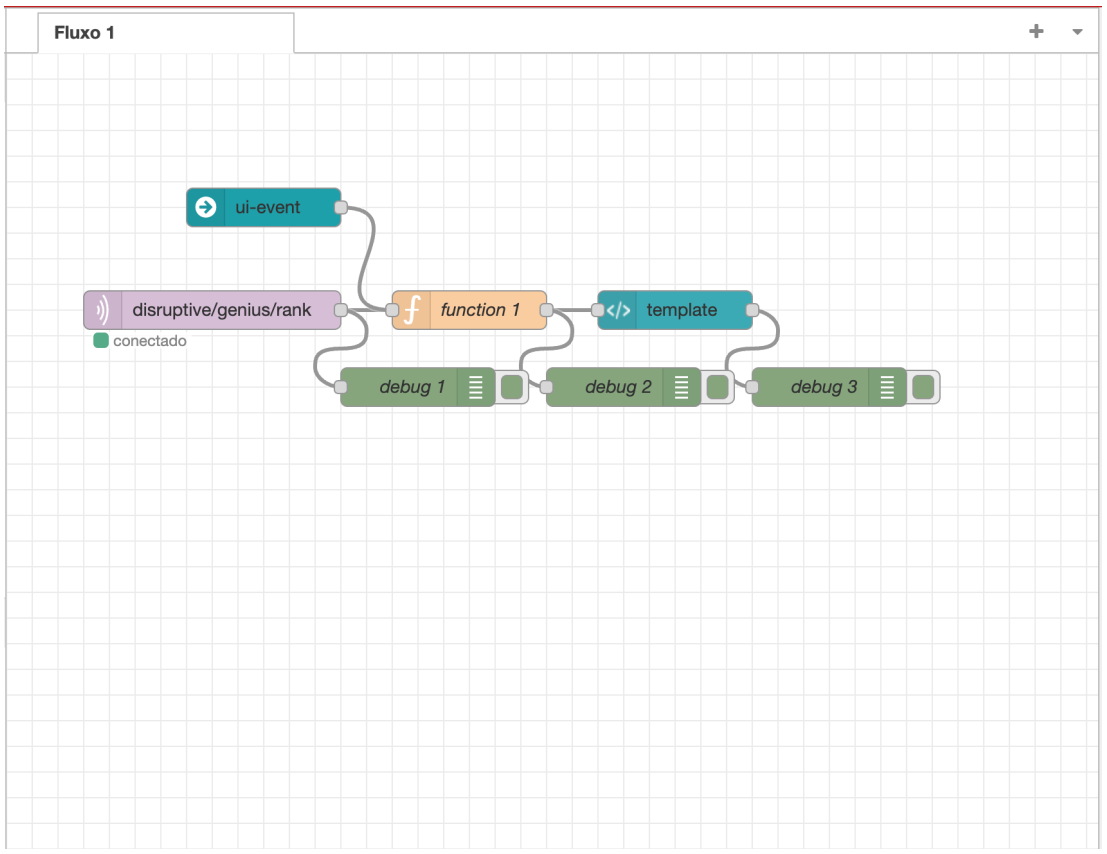
2. Lógica de programação: Nessa parte programamos em C o nosso projeto, para que ficasse e acordo com o que foi pedido pelo professor

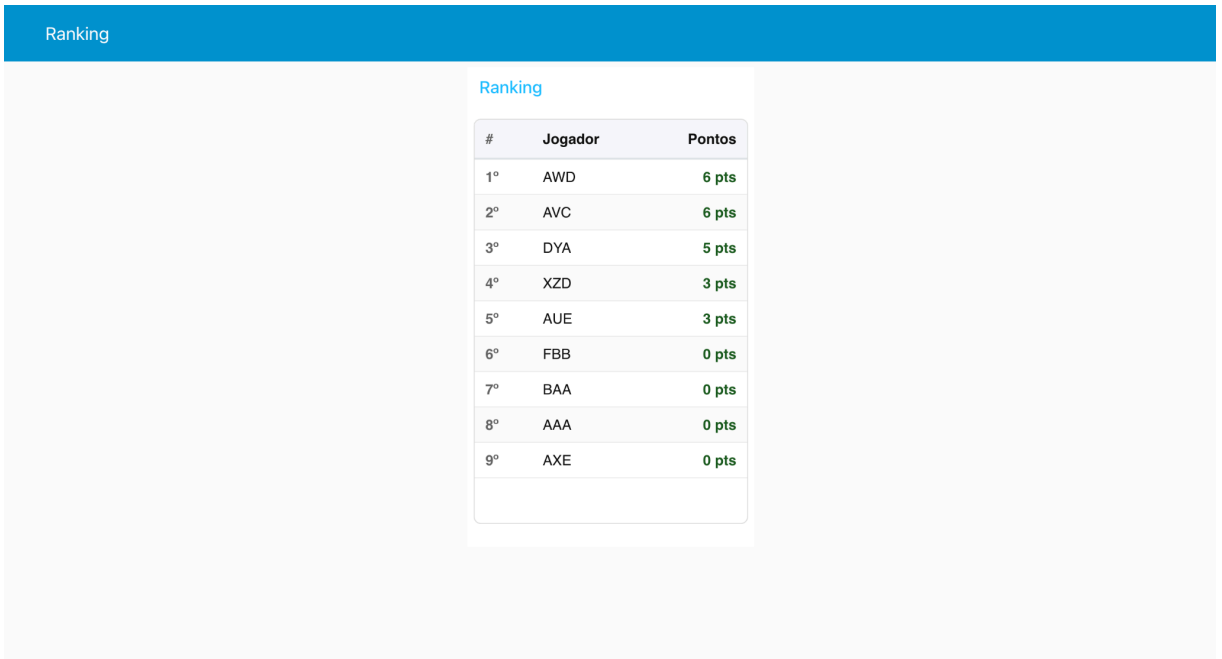
```
#include <WiFi.h>
#include <Wire.h>
#include <PubSubClient.h>
#include <LiquidCrystal_I2C.h>

const char* ssid = "Wokwi-GUEST";
const char* password = "";
const char* mqtt_server = "broker.hivemq.com";

WiFiClient espClient;
PubSubClient client(espClient);
LiquidCrystal_I2C lcd(0x27, 16, 2);

const int inverm = 12; const int inverd = 14; const int inazul = 27; const int inamar = 26;
const int outverm = 0; const int outverd = 4; const int outazul = 16; const int outamar = 17;
const int buzzer = 19;

typedef enum ESTADOS { STARTUP, TAG_SELECTION, GAME, GAMEOVER } estados;
estados estadoAtual;

int GameState = 0;
int cores[100];
int nivel = 0;
int atual = 0;
int keyDown = 0;
float tempoBase = 400.0;

char tag[4] = {'A', 'A', 'A', '\0'};
int tagIndex = 0;


void setup_wifi() {
  delay(10);
  lcd.clear();
  lcd.print("Conectando WiFi");
  WiFi.begin(ssid, password);
```

```
157  void runGameOver() {
162    publicarResultado();
163
164    tone(buzzer, 150, 1000);
165    delay(4000);
166
167    nivel = 0; atual = 0; GameState = 0; tagIndex = 0;
168    tag[0] = 'A'; tag[1] = 'A'; tag[2] = 'A';
169    estadoAtual = TAG_SELECTION;
170  }
171
172  void setup() {
173    Serial.begin(115200);
174    lcd.init(); lcd.backlight();
175    pinMode(inverm, INPUT_PULLDOWN); pinMode(inverd, INPUT_PULLDOWN);
176    pinMode(inazul, INPUT_PULLDOWN); pinMode(inamar, INPUT_PULLDOWN);
177    pinMode(outverm, OUTPUT); pinMode(outverd, OUTPUT);
178    pinMode(outazul, OUTPUT); pinMode(outamar, OUTPUT);
179    pinMode(buzzer, OUTPUT);
180    randomSeed(analogRead(34));
181    estadoAtual = STARTUP;
182  }
183
184  void loop() {
185    if (!client.connected() && estadoAtual != STARTUP) {
186      reconnect();
187    }
188    client.loop();
189
190    switch(estadoAtual) {
191      case STARTUP: runStartup(); break;
192      case TAG_SELECTION: runTagSelection(); break;
193      case GAME: runGame(); break;
194      case GAMEOVER: runGameOver(); break;
195    }
196  }
```

3. Conexão com o MQTT: Nessa parte utilizamos o MQTT para guardar os dados salvos do jogo na nuvem, ou seja a pontuação de cada jogador.

4.Inteligência e visualização(Node-Red): O node-red é como o cérebro do projeto completo, tivemos que utilizar os fluxos do MQTT para aplicar as abas no node, ele tem o intuito de comparar as pontuações.





#	Jogador	Pontos
1º	AWD	6 pts
2º	AVC	6 pts
3º	DYA	5 pts
4º	XZD	3 pts
5º	AUE	3 pts
6º	FBB	0 pts
7º	BAA	0 pts
8º	AAA	0 pts
9º	AXE	0 pts